# EF Core Migrations Guide

This document explains common database schema operations using Entity Framework Core migrations in a clean and structured way.

---

# 1. Install EF Tools

```
dotnet tool install --global dotnet-ef
```

Verify installation:

```
dotnet ef --version
```

---

# 2. Create Initial Migration (Create All Tables)

When models and DbContext are ready:

```
dotnet ef migrations add InitialCreate
```

Apply to database:

```
dotnet ef database update
```

---

# 3. Add New Table

### Step 1: Create Model

```
public class Project
{
    public int Id { get; set; }
    public string Name { get; set; } = null!;
}
```

### Step 2: Add DbSet

```
public DbSet<Project> Projects { get; set; }
```

### Step 3: Migration

```
dotnet ef migrations add AddProjectTable
dotnet ef database update
```

---

# 4. Add New Column

### Step 1: Update Model

```
public string PhoneNumber { get; set; }
```

### Step 2: Migration

```
dotnet ef migrations add AddPhoneNumberToEmployee
dotnet ef database update
```

---

# 5. Delete Table

### Step 1: Remove Model

### Step 2: Remove DbSet

### Step 3: Migration

```
dotnet ef migrations add RemoveProjectTable
dotnet ef database update
```

---

# 6. Rename Column (IMPORTANT)

### Step 1: Update Model

```
public string FullName { get; set; }
```

### Step 2: Create Migration

```
dotnet ef migrations add RenameNameToFullName
```

### Step 3: Edit Migration File

```
migrationBuilder.RenameColumn(
    name: "Name",
    table: "Employees",
    newName: "FullName");
```

### Step 4: Apply

```
dotnet ef database update
```

---

# 7. Seeding Data (HasData)

### Example

```
modelBuilder.Entity<Department>().HasData(
    new Department { Id = 1, Name = "HR" },
    new Department { Id = 2, Name = "IT" }
);
```

### Migration

```
dotnet ef migrations add AddSeedData
dotnet ef database update
```

---

# 8. Runtime Seeding (Recommended for real apps)

## apps)

```
if (!context.Departments.Any())
{
    context.Departments.AddRange(
        new Department { Name = "HR" },
        new Department { Name = "IT" }
    );

    context.SaveChanges();
}
```

Call in Program.cs using scoped service.

---

# 9. Golden Rule

Every schema change follows:

```
Model Change → Migration → Database Update
```

---

# 10. Important Commands Summary

```
# Create migration
dotnet ef migrations add MigrationName

# Update database
dotnet ef database update

# Remove last migration (if needed)
dotnet ef migrations remove
```

---

# End of Guide